User-Defined Functions in C

- There are library functions and user defined functions

- C functions like main(), scanf(), printf(), sqrt(), sin(), cos() etc. are already defined and user only uses them

- The code for user-defined function is to be written by the user, hence it is called user-defined function

Need for User-Defined functions

- main() is a special function in C where the execution starts
- It is possible to code any program using only main() function; but there are many disadvantages
- The program will be too large and complex, as a result of it debugging, testing and maintenance becomes difficult
- If the program is written using various functions for various parts of the program like for

  input, solution and output, each part can be independently coded and then combined into a single unit

Need for User-Defined functions contd..

- These independently coded parts of the program are called subprograms
- These subprograms are easier to understand, debug and test
- Such subprograms are called functions in C
- The second important advantage of using functions is

writing repeated code or writing the same code many times can be avoided

For example computation of a factorial or average of certain values is required at many places in a program

Need for User-Defined functions contd..

- The repeated code can be written as a function and the function can be called at many places

- Where ever the function name appears, it will be replaced with the code

- Thus using functions, program can be short, and easily understandable

Example using function

```
void printline(void);  /* function declaration */
main ( )
{
 printline ( );  /* calling the function*/
 printf("This demonstrates the use of C functions\n");
 printline ( );
}
void printline(void)    /* Defining the function */
 {
    int i;
   for (i=1; i<40; i++)
    printf("-");
    printf("\n");
 }
```

Output: ----------

This demonstrates the use of C functions

---

main( ) and printline( ) are the two user defined functions in the above program

Execution of a program using function

- The program execution always begins with the main( ) function
- During execution of main when a function call statement is encountered, control is transferred to function definition
- After execution of the function, control is transferred back to the main( ) and next statement to the function call is executed
- A function can call any other function; the called function can call another function and so on
- A function can call itself; such a function is known as a recursive function
- The functions can be placed anywhere in the program
- A called function code can be placed either before or after the calling function; it is usual practice to put all the called functions at the end

Components of a user-defined function

- There are three components

1. Function definition
2. Function call
3. Function prototype

- Function definition is written to implement the requirements of the function
- To use the function, we need to invoke it; this is referred to as calling the function, the program or function which calls a function is

referred to as calling program or calling function

- The calling program should declare any function

that is to be used later in the program; this is known as function declaration or function prototype

Definition of functions

- A function definition shall include the following
- function name
- function type
- list of parameters
- local variable declarations
- function statements and
- a return statement
- All the above six elements are grouped into two parts as

  function header (first three elements) and

  function body (last three elements)

General format of a function definition

```
function_type  function_name ( parameter list)
 {
   local variable declaration;
   executable statement1;
   executable statement2;
   ……….
   return statement;
}
```

Function parameters

- Formal Parameters: Function can take input through parameters to perform operations
- These parameters can also send output values to the calling program
- These parameters represent actual input values and also called formal parameters or formal arguments
- The parameter list contains declaration of variables separated by commas
- Examples: float quadratic (int a, int b, int c) { …}
- double power (double x, int n) { … }
- int sum (int a, int b) { … }

Function parameters contd..

- When there is no need of input for the function, void is used to indicate that the parameter list is empty
- Example: void printline(void) { …}
- It is a good programming style to use void to indicate a nil parameter list
- The function body is contained in {…}

  It includes local declarations, function statements and a return statement
- If a function does not return any value like in the above example, return statement can be omitted

Return values and their types

- A function may or may not send back any value to the calling function
- A function can return only one value per call, though it can take any number of input values
- The return statement can be

  return; which returns nothing or

  return (expression); returns the value of the expression

return (expression)

Example:

```
 int mul (int x, int y)
  {
     int p;
     p = x * y;
     return (p);    or combining two statements
    return (x * y);
   A function may have more than one return statement
    if ( x <= 0)
        return (0);
      else return (1);
```

Data type of function

- data type of return value is the data type of the function
- All functions by default return int type of data
- The data type is mentioned before the function name as below

```
int product (void)
    {
      return ( 2.5 * 3.0);
    }
```

Though product is float, the value 7 is returned

Function calls and actual arguments

- A function can be called by using the function name followed by actual parameters or arguments in parentheses
- Example

```
main ( )
{
   int y;
   y = mul (10, 5);  /* function call */
  printf (“ %d\n”, y);
}
```

in the function call above, the values 10 and 5 are called actual arguments and are assigned to the formal parameters and the value returned by the function is assigned to y

Different ways of calling a function

for example

mul (10, 5)

mul (m, 5)

mul (10, n)

mul (m, n)

mul (m+5, 10)

mul (10, mul (m, n))

mul (expression1, expression2)

Expressions must be evaluated to a single value

Using function in an expression

- function can be used in an expression like a variable
- printf (“%d\n”, mul (p,q));
- y = mul (p,q) / (p+q);
- if (mul (m,n) > sum) printf (“large”);

Important Note: the actual parameters must match the function’s formal parameters in type, order and number

Useful points about functions

- The parameter names need not be the same in prototype declaration and the function definition
- Use of parameter names in the declaration is optional as shown below
- If the function has no formal parameters, the list is written as void
- The return type is optional when the function returns int type data as shown below
- The return type must be void if no value is returned
- *int mul (int, int);*
- *mul (int a, int b);*
- *mul (int, int);*

Category of functions

- Depending on whether arguments are present or not, whether a value is returned or not, functions belong to one of the following categories
- Functions with no arguments and no return values
- Functions with arguments and no return values
- Functions with arguments and one return value
- Functions with no arguments but return a value
- Functions that return multiple values (by using call by reference)

Functions with Arguments and return values

- If a function does not send back any value, it displays the results of the calculation in the function itself
- But we may use the result of calculation in the calling function for further processing
- To have high degree of portability between programs, a function should be coded without involving any I/O operations
- A self contained and independent function receives a predefined form of input and returns a desired value

Example: Computation of GCD

```
#include<stdio.h>
int main()
{
void gcd(int a,int b);
int num1,num2;
printf("\n Enter 2 Numbers : ");
scanf("%d%d",&num1,&num2);
if(num1>num2)
{
gcd(num1,num2);
}
else
gcd(num2,num1);
return 0;
}
```

Function GCD

```
void gcd(int a,int b)
{
int g,rem;
rem=a%b;
if(rem==0)
{
g=b;
}
else
{
while(rem!=0)
```

Function GCD contd..

```
{
a=b;
b=rem;
rem=a%b;
}
g=b;
}
printf("\n GCD : %d",g);
}
```

Output: Enter 2 numbers : 35 3
GCD : 1

Computing factorial using function

```
/*Factorial  of a number */
/*Function calling a function*/
#include<stdio.h>
#include<conio.h>
void fact(int);  /* prototype declarations */
void call();
int main()
{
//clrscr();
call();
getch();
return 0;
}
void call()
{
        int num=0,i;
        printf("Enter number to find factorial: ");
        scanf("%d",&num);
    fact(num);
}
```

Function fact

```
void fact(int num)
{
    long int x,n=1;
    for(x=num;x>=1;x--)
    {
        n*=x;
    }
    printf("Factorial of %d is: %d \n",num,n);
}
```

Find sum of digits with recursive and non-recursive user defined functions

- **/* Find sum of digits with recursive and non-recursive user defined functions.*/**
- #include<stdio.h>
- int main()
- {
- int sum_uf(int num);
- int sum_recursive(int num);
- int n,i,sum1,sum2;
- printf("\n Enter number : ");
- scanf("%d",&n);
- sum1=sum_uf(n);
- sum2=sum_recursive(n);
- printf("\n Sum of digits using non-recursive function : %d",sum1);
- printf("\n Sum of digits using recursive function : %d",sum2);
- return 0; }
-

Function: sum of digits

- int sum_uf(int num)
- {
- int rem,sum=0;
- while(num!=0)
- {
- rem=num%10;
- num=num/10;
- sum=sum+rem;
- }
- return sum;
- }
- int sum_recursive(int num1)

Recursive Function: sum of digits

```
int sum_recursive(int num1)
{
int rem=0;
if(num1==0)
return 0;
else
{
rem=rem+num1%10;
return rem+sum_recursive(num1/10); // recursive call
}
}
```

Output: Sum of digits

```
Enter number : 459

Sum of digits using non-recursive function : 18
Sum of digits using recursive function : 18
```

Call by value and call by Reference

- Call by value means sending the values of the arguments to function
- When a value is passed to a function via an actual argument, the value of the actual argument is copied into the corresponding formal argument
- Thus the value of the formal argument can be altered within the function, but the value of the actual argument within the calling function will not change
- This process of passing the value of an argument to a function is known as passing by value or call by value

Example: Call by value

- /* The function in this program changes the value of its argument */
- #include <stdio.h>
- main()
- { int a =2;
- printf("\na = %d from main before calling the function", a);
- modify (a);
- printf ("\na = %d from main, after calling the function", a);
- }

Example: call by value contd..

- modify(int a)
- {
- a * = 3;
- printf("\na = %d from the function, after being modified", a);
- return;
- }
- Output: a =2 from main before calling the function
- a=6 from the function, after being modified
- a =2 from main, after calling the function

Observe that **a** is not altered within the main, even though the corresponding value of **a** is changed in the function

Call by Reference

- Call by reference means sending the address of the arguments to the called function
- The address of actual arguments in the calling function are copied into formal arguments of the called function
- Using these addresses, the actual arguments are accessed and whatever changes take place in the function are performed on the actual arguments also
- For accessing the address of variable, pointers are used

Passing Arrays to Functions

- In call by value, we pass values of array elements to the function
- in call by reference, the name of the array without subscript and the size of the array are passed to the function
- In C, the name of the array represents the address of the first element
- By passing the array name, we are passing the address of the array to the called function, and hence the array refers to the same array stored in the memory
- Any changes in the array in the called function will be reflected in the original array

Example: Pass by Value

```
 #include<stdio.h>
void display(int m);
  int main()
    {
      int i;
   int num[ ] = {35, 40, 45, 50, 55};
    for (i =0; i<=4; i++)
      display(num[i]);
      return(0);
   }
  void display(int m)
  {
    printf("\n%d", m);
  }
```

Example: Pass by Reference with explicit pointer declaration

```
#include <stdio.h>
void display(int*, int);
int main()
      {
         int i;
 int num[] = {35, 40, 45, 50, 55};
         //for (i =0; i<=4; i++)
          display (num,5);
          return(0);
     }
 void display(int *num, int n)
 {
    int i;
    for (i =0; i<=n-1; i++)
    printf ("\n%d", num[i]);
     }
```

Call by Reference without explicit pointer declaration

```
#include <stdio.h>
 void display(int num[], int n);
int main()
      {
          int i;
  int num[] = {35, 40, 45, 50, 55};
          //for (i =0; i<=4; i++)
           display (num,5);
           return(0);
     }
  void display(int num[], int n)
  {
          int i;
    for (i =0; i<=n-1; i++)
    printf ("\n%d", num[i]);
           }
```

find out maximum and minimum element from an array

```
/* find out maximum and minimum element from an array */
#include<stdio.h>
#include<conio.h>
int main()
{
void maxmin(int ar[],int size);
int num[5]={23,54,12,45,84},i;
system("cls");  // clrscr()
printf("\n Array is : ");
for(i=0; i<5; i++)
        printf("\n %d",num[i]);
maxmin(num,5);  // call to function
getch();
return 0;   }
```

Function maxmin

```
void maxmin(int ar[],int size)    // finding max and min
{
int max=ar[0],min=ar[0],i;
        for(i=1; i<size; i++)
        {
                if(max<ar[i])
                max=ar[i];
                if(min>ar[i])
                min=ar[i];
        }
        printf("\n Max : %d     Min : %d",max,min);
}
```

Matrix operations using functions

```
/* perform addition , subtraction and multiplication of two matrices
*/
#include<stdio.h>
#include<conio.h>
void print(int mat[][2],int row,int col);
int main()
{
void addition(int mat1[][2],int mat2[][2],int mat3[][2],int row,int col);
void subtraction(int mat1[][2],int mat2[][2],int mat3[][2],int row,int col);
void multiplication(int mat1[][2],int mat2[][2],int mat3[][2],int row,int col);
int mat1[2][2]={1,2,3,4},mat2[2][2]={1,2,3,4},mat3[2][2];
system("cls");
print(mat1,2,2);
print(mat2,2,2);
addition(mat1,mat2,mat3,2,2);
subtraction(mat1,mat2,mat3,2,2);
multiplication(mat1,mat2,mat3,2,2);
getch();
return 0;
```

Matrix operations using functions contd..

```
}
void print(int mat[][2],int row,int col)
{
        int i,j;
        printf("\n Array is : \n");
        for(i=0; i<row; i++)
        {
                for(j=0; j<col; j++)
                {
                        printf(" %d ",mat[i][j]);
                }
        printf("\n");
        }
}
```

Matrix operations using functions contd..

```
void addition(int mat1[][2],int mat2[][2],int mat3[][2],int row,int col)
{
int i,j;
        for(i=0; i<row; i++)
         for(j=0; j<col; j++)
        mat3[i][j]=mat1[i][j]+mat2[i][j];
        printf("\n Addition Done!");
print(mat3,2,2);
}
void subtraction(int mat1[][2],int mat2[][2],int mat3[][2],int row,int col)
{
int i,j;
        for(i=0; i<row; i++)
                for(j=0; j<col; j++)
        mat3[i][j]=mat1[i][j]-mat2[i][j];
        printf("\n Subtraction Done!");
print(mat3,2,2);
}
```

Matrix operations using functions contd..

```
void multiplication(int mat1[][2],int mat2[][2],int mat3[][2],int row,int col)
{
        int i,j,k;

for(i=0; i<row; i++)
        {
                for(j=0; j<col; j++)
                {
                mat3[i][j]=0;
                        for(k=0; k<col; k++)
                        {
                        mat3[i][j]=mat3[i][j]+mat1[i][k]*mat2[k][j];
                        }
                }
        }
        printf("\n Multiplication Done!");
print(mat3,2,2);
}
```

RECURSION

- Recursion is an important case of procedure abstraction where the abstraction that a procedure is built on is the procedure itself.
- Procedure abstraction helps to think at a higher level by allowing to suppress the details of how things are done at lower level.

RECURSION (Contd..)

- Procedure abstraction allows to write procedures into layers, such that the higher level definition is unaffected by the definition of lower layers, thus making procedure definition small and easy to understand.

RECURSION (Contd..)

Recursion allows procedures or functions to use themselves

$m^n = m \times m^{n-1}$ for $n > 0$

$= 1$ if $n = 0$

$n! = n \times (n - 1)!$ for $n \geq 1$

$= 1$ for $n = 0$

RECURSION contd..

- It is a powerful programming technique.
- Recursive programs may need more execution (than corresponding iterative ones) on some compilers.
- Direct Recursion- when a procedure includes a call to it self.
- Indirect Recursion- when a procedure calls another procedure which then causes the original procedure to be reinvoked

Examples of Recursive programs

1. Fibonacci sequence 0,1,1,2,3,5,8,13,21,34,…

   $F_0 = 0$, $F_1 = 1$, $F_i = F_{i-1} + F_{i-2}$, i.>1

   Procedure F(n)

   // returns the nth Fibonacci number//

   integer n

   if n<=1 then return (n)

   else return (F(n-1) + F(n-2))

   endif

   end F

Examples of Recursive programs contd..

2. Searching for position of X in array A (1:n)

```
  procedure search (i).
//Searches for the first K such that A (K)=X in A (i:n)//
//Otherwise 0 is returned//
Global n, x, A (1:n)
Case : i > n: return (0)
          : A (i) = x : return (i)
         : else : return (Search ( i+1)
          : end case
            end Search
```

Examples of Recursive programs contd..

Explanation

Search (1) is called first

Search (1) returns k such that A (K)=x in A(1 : n)

Search (*I*)returns K such that A(K)=x in A(I : n)

If A (i)=x and i<n, return i.

If A (i)<>x and i< n then call search (i+1)

If A (i)<>x and i.> n then return 0

Examples of Recursive programs contd..

3. Euclid algorithm for GCD

```
procedure GCD (a,b)
        //Assume a > b>0//
//If a <b then in the next step GCD (a,b) becomes GCD
        (b,a)//
        If b=0 then return (a) //so the condition a>b reduces
        the number of steps//
        else return (GCD ( b,a mod b ))
end GCD.
```

RECURSION AND ITERATION

- To translate a recursive procedure into an equivalent iterative one, stack is used to hold the values of parameters, local variables, and function value.
- A stack is a linear data structure in which the last item stored will be the first one to be removed (LIFO)
- Label is attached to executable statements.
- Each recursive call is replaced by a set of instructions.
- The computing time of iterative program is less than or equal to the corresponding recursive program.

RECURSION AND ITERATION contd..

A stack is not needed if the last statement of a procedure is a recursive call as in Euclid algorithm for GCD.

To convert it into an iterative one evaluate the new values of the parameters and branch the control to the beginning.

"GCD with recursion"procedure GCD (a,b) "a>b>0"if b=0 then return (a)else return GCD (b,a mod b) endif end GCD	GCD with iteration Using while do// Procedure GCD2 (a,b) while b<>0 do t←b;b←a mod b;a←t repeat return (a) End GCD2	//GCD using if then else// Procedure GCD3 (a,b) L1: if b=0 then return (a) else t←b b← a mod b a←t go to L1 endif end GCD3

Find sum of first n numbers using recursion

.

```
#include<stdio.h>
int main()
{
int sum(int num);
int n,ans;
printf("\n Enter n : ");
scanf("%d",&n);
ans=sum(n);
printf("\n Sum of %d first numbers : %d",n,ans);
return 0;
}
int sum(int num)
{
if(num==0)
return num;
else
{
return num=num+sum(num-1);
}
}
```

```
Enter n : 5

Sum of 5 first numbers : 15
```

Find a power b using recursive function

```
#include<stdio.h>
int main()
{
int pow_rec(int a,int b);
int a,b,ans;
printf("\n a and b (a power b) : ");
scanf("%d%d",&a,&b);
ans=pow_rec(a,b);
printf("\n Ans : %d",ans);
return 0;
}
int pow_rec(int a,int b)
{
if(b==0)
return 1;
else
return a=a*pow_rec(a,b-1); }
```

```
a and b (a power b) : 5 3

Ans : 125
```